

|                                                                                   |                                    |  |
|-----------------------------------------------------------------------------------|------------------------------------|--|
|  | Carátula para entrega de prácticas |  |
| Facultad de Ingeniería                                                            | Laboratorio de docencia            |  |

# Laboratorios de computación salas A y B

*Profesor(a):* Ariel Adara Mercado Martinez

*Asignatura:* Estructura de Datos y Algoritmos I

*Grupo:* 5

*No. de práctica(s):* 4

*Integrante(s):* Roque Robledo Fernando Ignacio

Jasso Olivares Germán

Hernandez Piedra José Enrique

Hernández Beltran Ena Elianeth

*No. de lista o brigada:* 6

*Semestre:* 2do semestre

*Fecha de entrega:* 12/Marzo/2026

## ÍNDICE

|                                                         |          |
|---------------------------------------------------------|----------|
| <b>Introducción y Objetivos.....</b>                    | <b>2</b> |
| <b>Estructuras de Datos Utilizadas.....</b>             | <b>3</b> |
| <b>Algoritmo de Conversión (Infija a Postfija).....</b> | <b>4</b> |
| <b>Evaluación de Expresiones Postfijas.....</b>         | <b>5</b> |
| <b>Casos de Prueba y Validaciones.....</b>              | <b>5</b> |
| <b>Conclusiones.....</b>                                | <b>6</b> |
| <b>Bibliografía.....</b>                                | <b>6</b> |

## OBJETIVOS

Diseñar e implementar un sistema modular en lenguaje C que sea capaz de recibir una expresión algebraica en notación convencional (infija), transformarla rigurosamente a notación postfija mediante el uso de colas y pilas dinámicas basadas en listas ligadas, y calcular su resultado numérico exacto con la evaluación de la estructura que resultó.

## INTRODUCCIÓN

Las operaciones matemáticas y algebraicas están comúnmente escritas en notación fija, es decir una convención matemática en donde los operadores se posicionan estrictamente entre los operandos que afectan (ejemplo,  $x + y$ ). A pesar de la facilidad de su lectura, esta notación tiene grandes complicaciones para el procesamiento automático en sistemas de computadoras o lenguaje computacional, ya que necesita del análisis recursivo de reglas de precedencia, agrupaciones explícitas expresadas en paréntesis y un escaneo continuo de la cadena de texto hacia adelante y hacia atrás.

Para ayudar a solventar dichos problemas de incidencia, con ayuda de la computación se recurre a la notación postfija o notación polaca inversa. En este ordenamiento, los operadores se colocan después de sus operandos correspondientes. Después de eliminar por completo la necesidad de paréntesis de agrupación y fijar un orden de ejecución estrictamente lineal de izquierda a derecha, se optimiza drásticamente el uso de los ciclos del procesador.

### 2. Estructuras Utilizadas

Para evitar las limitaciones de tamaño fijo, todo el software utilizó las listas ligadas dinámicas. Lo que permite que la memoria crezca o disminuya en tiempo de ejecución conforme a la longitud de la expresión ingresada.

#### 2a. Pila

Es una estructura lineal que opera bajo el comportamiento LIFO (*Last In, First Out*). Las operaciones atómicas desarrolladas fueron:

- Push: Reserva un nuevo nodo en memoria e intercepta el "tope" de la pila.
- Pop: Extrae el elemento situado en el tope actual, redirige el puntero de control al nodo subsecuente y libera la memoria del nodo retirado.
- Peek: Permite examinar el valor del operando u operador en el tope sin alterar el estado

de la pila.

## **2b. Cola**

La cola es una estructura lineal regida que se implementó manteniendo un doble control mediante dos punteros de referencia (uno apuntaba al inicio y el otro al final)

- Enqueue: Inserta nuevos elementos exclusivamente por el extremo trasero de la lista ligada.
- Dequeue: Remueve y recupera elementos partiendo únicamente del extremo frontal de la estructura.

En esta práctica la cola se usó como el contenedor definitivo para alojar la expresión postfija ya ordenada. Al preservar el orden de inserción de los elementos, así la lectura en la etapa de evaluación se llevará de manera exacta.

## **2C. Memoria Dinámica**

Toda asignación de memoria se trabajó mediante la función malloc(), reservando espacio para las estructuras de los nodos de la lista ligada. Con el fin de asegurar una ejecución limpia y así prevenir “fugas” en la memoria, se estructuraron rutinas de limpieza utilizando la función free(), las cuales liberan secuencialmente cada bloque asignado una vez que los datos han cumplido su ciclo de vida en el algoritmo.

## **3. Algoritmo de Conversión**

La transformación conceptual se rige por el comportamiento matemático de los operadores a través de dos reglas de ordenación:

### **3a. Precedencia**

Clasifica el nivel jerárquico de ejecución. El operador de potencia (wedge) ostenta la jerarquía más alta; seguido por la multiplicación y división (\*, /) con nivel medio; y finalmente la suma y resta (+, -) en el peldaño inferior.

### **3b. Asociatividad**

Dictamina la dirección de lectura cuando coinciden operadores con la misma jerarquía. La mayoría posee asociatividad por la izquierda, a excepción del operador de potencia (wedge), el

cual se evalúa de derecha a izquierda.

### 3c. Funcionamiento del Algoritmo

Infija: La notación infija es la representación algebraica tradicional en la cual los operadores se encuentran entre operandos.

postfija: La notación postfija coloca los operadores después de los operandos.

## 5. Casos de Prueba y Validaciones

Para garantizar la fiabilidad del código ante condiciones de estrés algebraico, se corrieron múltiples escenarios de validación:

Jerarquías Complejas:

- Entrada Infija:

$$x - y / z$$

- Cola Postfija Resultante:

$$x y z / -$$

Control de Asociatividad por la Derecha:

- Entrada Infija:

$$a \wedge b \wedge c$$

- Cola Postfija Resultante:

$$a b c \wedge \wedge$$

Agrupación por Paréntesis:

- Entrada Infija:

$$(a + b) * (c - d)$$

- Cola Postfija Resultante:

$$a b + c d - *$$

**Validaciones del Sistema:** El programa cuenta con un robusto bloque de control de errores para evitar fallas catastróficas en el sistema (*crashes*):

Verificación de Punteros Nulos: Cada llamada a `malloc()` es validada inmediatamente; si el sistema se queda sin memoria, el programa aborta de forma segura notificando el error.

**Validación de Paréntesis:** Si al terminar la conversión quedan residuos de paréntesis en la pila o si falta un elemento de apertura, se despliega un mensaje de error ("Expresión mal balanceada").

División por Cero: Durante la fase de evaluación, antes de procesar el operador `/`, se valida si el segundo operando es exactamente 0, interrumpiendo la operación para evitar una falla de segmentación.

## Conclusiones

Dificultades Encontradas: El reto técnico más complejo residió en la correcta implementación de la asociatividad por la derecha para la potencia. Inicialmente, el algoritmo trataba de igual manera todos los operadores, lo que rompía el flujo matemático estándar. Asimismo, asegurar que el orden de los operandos al hacer *Pop* en la división o resta no se invirtiera requirió de un mapeo lógico minucioso.

Decisiones de Diseño: Separar las estructuras en archivos independientes (`pila.c`, `cola.c`, `postfija.c`, `main.c`) demostró ser la mejor estrategia de diseño. Esto facilitó el aislamiento y depuración de errores específicos en la gestión de los punteros sin afectar las rutinas lógicas de la conversión matemática.

Aprendizaje Obtenido: El desarrollo práctico de este software permitió asentar de forma definitiva el entendimiento de la memoria dinámica en C. La abstracción de conceptos complejos como pilas y colas cobró un significado real al ver cómo estructuras de datos tan elegantes pueden simplificar un problema de interpretación de lenguaje humano en una secuencia directa y eficiente para el procesador de una computadora.

## Bibliografía

- Tenenbaum, Aaron M. *Estructuras de datos en C*. Prentice Hall.
- Cairó, Osvaldo. *Estructuras de Datos*. McGraw-Hill.

- Manual de Prácticas de Estructuras de Datos y Algoritmos I, Facultad de Ingeniería, UNAM.